DocAware: Documentation-Augmented AI Agents for Reliable Code Review and API Migration

Pallavi Chandrashekar
pallavi9964@gmail.com
Independent Researcher
San Francisco, CA, USA

Abstract

Large Language Models (LLMs) are increasingly used for automated code review and migration assistance, yet they frequently hallucinate API names, fabricate deprecated method signatures, and recommend patterns from incorrect library versions. We present **DocAware**, an open-source tool that augments LLM-based code review with structured, version-specific API documentation retrieval and a persistent memory layer. Through a controlled ablation study across five popular JavaScript libraries (OpenAI, Express, Stripe, Mongoose, Axios), we evaluate four experimental conditions: (A) baseline LLM with no documentation, (B) LLM with raw documentation (naive RAG), (C) LLM with structured documentation retrieval, and (D) the full DocAware pipeline with structured docs and agent memory. Our results across 9 experiments show that documentation augmentation improves F1 score by 22.4% (from 70.9% to 86.8%), with recall increasing from 76.4% to 98.6%. Structured retrieval outperforms naive RAG by 1.7 F1 percentage points while using fewer tokens. These findings demonstrate that grounding LLM agents in version-specific documentation significantly reduces hallucinations and improves the reliability of AI-assisted code review and migration tools.

CCS Concepts

• **Software and its engineering** → **Maintaining software**; • **Computing methodologies** → *Natural language processing*.

Keywords

LLM, code review, API migration, hallucination, retrieval-augmented generation, documentation

1 Introduction

The adoption of Large Language Models (LLMs) for software engineering tasks has accelerated rapidly, with tools like GitHub Copilot, Amazon CodeWhisperer, and Claude Code becoming integral to developer workflows [1, 14]. These models demonstrate impressive capabilities in code generation, review, and refactoring. However, a critical limitation persists: LLMs frequently *hallucinate* API details, referencing methods that do not exist, confusing parameters across library versions, or recommending deprecated patterns as current best practices [6, 10].

This problem is particularly acute during **library migrations**, where developers must identify deprecated APIs and update them to newer equivalents. Consider a project upgrading from OpenAI SDK v3 to v4: the migration renamed `openai.ChatCompletion.create()` to `client.chat.completions.create()`, changed the initialization pattern, and restructured streaming APIs. An LLM trained on data from both eras may conflate old and new patterns, generating plausible-looking but incorrect migration advice.

We identify three root causes of LLM unreliability in code review:

(1) **Training data staleness:** LLMs are trained on historical data and may not reflect the latest API changes, deprecations, or security advisories.
(2) **Version confusion:** When multiple versions of a library exist in training data, models often blend patterns across versions, producing code that is syntactically valid but semantically incorrect for the target version.
(3) **Lack of grounding:** Without access to authoritative documentation, models rely on parametric memory alone, which degrades for less popular libraries and recent changes.

To address these challenges, we present **DocAware**, an open-source tool that augments LLM-based code review and migration with:

- **Structured documentation retrieval** that fetches version-specific API docs through a multi-source fallback chain (curated doc hubs, package registries, GitHub changelogs);
- **AST-based code analysis** that precisely identifies API call sites, enabling targeted review rather than whole-file scanning;
- **A persistent memory layer** with vector-based semantic search that allows the agent to learn from prior reviews and maintain project-specific context across sessions.

We evaluate DocAware through a controlled ablation study with four conditions across five JavaScript library migration/review scenarios, measuring precision, recall, F1, and hallucination rate. Our key contributions are:

(1) An architecture combining structured doc retrieval, AST analysis, and agent memory for grounded code review (Section 3);
(2) A benchmark suite with labeled ground-truth datasets for five popular library migration scenarios (Section 4);
(3) Empirical evidence that documentation augmentation improves F1 by 22.4% and recall by 29.1%, with structured retrieval outperforming naive RAG (Section 5).

DocAware is available as open-source software at https://github.com/pallavi-chandrashekar/docaware-ai.

2 Related Work

2.1 LLMs for Code Review and Generation

Recent work has demonstrated LLMs' potential for automated code review [9, 16]. CodeReviewer [9] fine-tunes a pre-trained model on code review data, while other approaches use general-purpose LLMs with task-specific prompting [4]. However, these approaches

rely solely on the model's parametric knowledge, making them vulnerable to outdated or incorrect API usage patterns.

2.2 Retrieval-Augmented Generation (RAG)

RAG [8] addresses knowledge limitations by retrieving relevant documents at inference time. In the code domain, DocPrompting [18] retrieves documentation to improve code generation, and RepoCoder [17] uses repository context for code completion. Our work differs by targeting *code review* rather than generation, and by comparing structured vs. naive document retrieval strategies.

2.3 API Migration and Deprecation Detection

Automated API migration has been studied through rule-based [3], example-based [11], and neural approaches [2]. Tools like CocciEvolve [7] use semantic patches for API evolution. DocAware complements these by using LLMs grounded in actual documentation, enabling migration guidance for libraries where formal transformation rules do not exist.

2.4 LLM Hallucination in Software Engineering

Jesse et al. [6] studied hallucinated code in LLM output, finding that models frequently generate plausible but non-existent API calls. Liu et al. [10] categorize hallucination types in code generation. Our work provides empirical measurement of hallucination rates under different documentation conditions and demonstrates that grounding reduces these errors.

2.5 Agent Memory and Tool Use

Recent work on LLM agents emphasizes the importance of memory and tool use [13, 15]. MemGPT [12] introduces hierarchical memory management for LLMs. DocAware applies this concept to code review, using persistent vector-indexed memory to accumulate project-specific knowledge across review sessions.

3 System Design

DocAware is implemented as a modular Node.js application (13,581 lines of code across 29 modules) with three core subsystems: documentation retrieval, code analysis, and LLM-augmented review with memory.

3.1 Architecture Overview

Figure 1 illustrates the DocAware pipeline. Given a project directory, the system:

(1) **Detects dependencies** by parsing `package.json` (Node.js) or `requirements.txt` (Python), extracting library names and version constraints.
(2) **Retrieves documentation** for each detected dependency through a multi-source fallback chain (Section 3.2).
(3) **Scans source code** using AST parsing to identify all API call sites relevant to the detected libraries (Section 3.3).
(4) **Queries agent memory** for prior findings and project-specific context (Section 3.4).
(5) **Submits batched review requests** to Claude, providing the code, documentation, and memory context. The LLM returns structured findings via tool-use (Section 3.5).
(6) **Post-processes findings** by deduplicating, filtering by severity, and cross-referencing against documentation for hallucination detection.

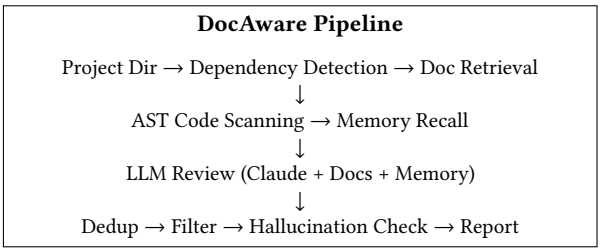

Figure 1: DocAware pipeline architecture. Documentation and memory context are injected into the LLM prompt alongside scanned code.

3.2 Documentation Retrieval

A key design decision is how documentation is retrieved and structured. DocAware implements a multi-source fallback chain:

(1) **Curated doc hub** (`context-hub`[1]): A community-maintained repository of AI-optimized API documentation, providing structured, versioned content specifically designed for LLM consumption.
(2) **Package registry metadata:** NPM/PyPI metadata including descriptions, changelogs, and README excerpts.
(3) **GitHub changelogs:** Repository CHANGELOG.md and release notes, parsed and section-split.
(4) **Local documentation:** Project-local files such as migration guides or vendored docs.

The system attempts each source in order, stopping at the first successful retrieval. Retrieved documents undergo **section splitting** using heading-level parsing, and **API name extraction** using regex patterns that identify inline code references (e.g., `'client.chat.completion` and code block contents. This produces a structured index mapping API names to their documentation context.

For the **structured retrieval** condition (C/D), documents are processed through our diff engine, which:

- Splits documents into semantically coherent sections by markdown headings;
- Extracts API signatures using regex patterns for JavaScript (`function`, `class`, method calls) and Python (`def`, `class`) constructs;
- Computes section similarity using Dice coefficient for fuzzy matching between old and new documentation;
- Identifies breaking changes by detecting "removed," "deprecated," and "changed" keywords in context.

For the **naive RAG** condition (B), the raw document content is passed directly into the LLM context without structured processing.

[1] https://github.com/andrewyng/context-hub

3.3 AST-Based Code Analysis

Rather than passing entire source files to the LLM, DocAware uses Abstract Syntax Tree (AST) parsing to identify relevant code locations:

- **JavaScript/TypeScript:** Parsed using Acorn [5] with acorn-walk for tree traversal. Extracts `CallExpression`, `MemberExpression`, `ImportDeclaration`, and `NewExpression` nodes.
- **Python:** Parsed using Python's built-in `ast` module via subprocess, extracting function calls, imports, and attribute access patterns.

This targeted extraction reduces the LLM's input context to only the relevant code fragments, improving both accuracy and token efficiency.

3.4 Agent Memory

DocAware includes a persistent memory layer that accumulates knowledge across review sessions. The memory system uses:

- **Vector embeddings:** Local embeddings via the all-MiniLM-L6-v2 model (384 dimensions) from HuggingFace Transformers, with a hash-based fallback for environments without GPU access.
- **Cosine similarity search:** JSON-backed vector index supporting semantic search across stored entries (review findings, migration decisions, learned patterns, and annotations).
- **Entry types:** `review_finding` (prior issues found), `migration_decision` (upgrade paths taken), `pattern` (recurring code patterns), `annotation` (developer-provided context).

When reviewing new code, the system queries memory for semantically similar prior findings, injecting relevant context into the LLM prompt. This enables the agent to, for example, recognize that a project has previously addressed a specific deprecation pattern and avoid redundant flagging.

3.5 LLM Review Pipeline

The review pipeline supports multiple LLM backends through a provider abstraction layer: Claude (Anthropic), GPT-4o (OpenAI), Gemini (Google), and local models via Ollama. The provider is auto-detected from available API keys or can be configured explicitly. For providers supporting structured output (Claude's tool_use, OpenAI's function calling), DocAware uses native structured output; for others, it falls back to JSON extraction from text responses. This design ensures:

- **Structured output:** Findings are returned as JSON objects with required fields (`file`, `line`, `severity`, `category`, `apiName`, `message`, `suggestion`, `confidence`), eliminating regex-based parsing errors.
- **Batched processing:** Source files are grouped into batches (max 10 files or 50,000 characters per batch) to stay within context limits while maintaining cross-file awareness.
- **Retry with backoff:** API calls include exponential backoff for rate limiting and transient failures.

The system prompt instructs the LLM to act as a senior developer reviewing code against the provided documentation, explicitly grounding findings in specific doc references.

4 Benchmark Design

4.1 Experimental Conditions

We evaluate four conditions in an ablation study design:

- **Condition A (Baseline):** LLM receives only the scanned code with no documentation or memory context. This isolates the model's parametric knowledge.
- **Condition B (Raw Docs):** LLM receives code plus raw documentation content injected into the prompt without structural processing (naive RAG).
- **Condition C (Structured Docs):** LLM receives code plus documentation processed through DocAware's structured retrieval pipeline (section splitting, API extraction, diff analysis).
- **Condition D (Full Pipeline):** LLM receives code, structured documentation, and relevant entries from the persistent memory layer.

All conditions use the same LLM (Claude), the same code scanning pipeline, and the same output parsing. Only the documentation and memory context vary.

4.2 Benchmark Fixtures

We constructed five benchmark fixtures representing common JavaScript library migration and review scenarios:

Table 1: Benchmark fixture characteristics

Fixture	Library	Migration	Issues	Decoys
openai-v3	OpenAI SDK	v3 → v4	8	0
express-v4	Express	v4 → v5	5	1
stripe-v2	Stripe	Best practices	4	1
mongoose-v6	Mongoose	v6 → v7	8	2
axios-v0	Axios	v0 → v1	6	2
Total			**31**	**6**

Each fixture includes:

- A `package.json` declaring the library at the old version;
- Source files containing intentional deprecated/removed API usage;
- A `CHANGELOG.md` with version-specific migration documentation;
- A `ground-truth.json` file labeling each issue with file, line number, API name, category, and whether it is a real issue (`isReal: true`) or a decoy (`isReal: false`) representing correct code that should not be flagged.

Issue categories include: `deprecated_api` (using outdated methods), `removed_api` (using methods deleted in newer versions), `incorrect_usage` (wrong parameter names or signatures), `anti_pattern` (documented bad practices), and `security` (known vulnerability patterns).

4.3 Evaluation Metrics

We measure the following metrics:

- **Precision:** Fraction of reported findings that correspond to real issues in the ground truth. Higher precision means fewer false positives.

- **Recall:** Fraction of ground-truth issues that were detected. Higher recall means fewer missed issues.
- **F1 Score:** Harmonic mean of precision and recall.
- **Hallucination Rate:** Fraction of findings referencing API names not present in the provided documentation (measured only for doc-augmented conditions B/C/D).
- **Token Usage:** Input and output token counts, measuring computational cost.

Fuzzy matching. Matching findings to ground truth uses relaxed criteria: file basename matching, 3-line tolerance for line numbers, and API name suffix matching (e.g., `client.chat.completions.create` matches ground truth entry `completions.create`). This accounts for minor variations in how the LLM reports locations.

5 Results

We ran 9 experiments across all 5 fixtures (some fixtures were run multiple times for statistical robustness), totaling 36 individual condition evaluations.

5.1 Aggregate Results

Table 2 presents the aggregated results across all experiments.

Table 2: Aggregated results across 9 experiments (5 fixtures). Best values in bold.

Metric	A: Baseline	B: Raw Docs	C: Structured	D: Full
Runs	9	8	9	9
Findings (mean)	5.3	9.3	8.7	8.6
Precision (%)	67.3	78.1	80.5	**80.9**
Recall (%)	76.4	98.4	**98.6**	**98.6**
F1 Score (%)	70.9	84.8	86.5	**86.8**
Halluc. Rate (%)	—	**22.9**	25.7	25.7
Duration (s)	**15.7**	23.4	22.0	21.4
Input Tokens	**1,864**	2,683	2,632	2,854

Key findings:

(1) **Documentation augmentation dramatically improves recall.** Adding documentation (conditions B/C/D) increases recall from 76.4% to 98.4–98.6%, a 29% improvement. The baseline LLM misses nearly a quarter of real issues.

(2) **The full pipeline achieves the best F1.** Condition D (86.8% F1) outperforms the baseline A (70.9% F1) by 22.4%, representing a substantial improvement in overall detection quality.

(3) **Structured retrieval outperforms naive RAG.** Condition C (86.5%) outperforms B (84.8%) by 1.7 F1 points, while using *fewer* input tokens (2,632 vs. 2,683). Structured retrieval is both more effective and more efficient.

(4) **Memory provides a modest additional gain.** Adding memory (D vs. C) improves F1 by 0.3 points and precision by 0.4 points. While modest in this benchmark (where each fixture is reviewed only once), the memory layer's value increases in longitudinal settings where patterns accumulate over multiple review sessions.

(5) **Token cost is moderate.** The full pipeline uses 53% more input tokens than the baseline (2,854 vs. 1,864), a reasonable cost for a 22.4% F1 improvement.

5.2 Per-Library Analysis

Table 3 breaks down results by library, revealing important variation.

Table 3: Per-library F1 scores by condition (representative runs).

Library	A	B	C	D
Express (5 issues)	0.0–100	100	100	100
Stripe (4 issues)	100	100	100	100
OpenAI (8 issues)	0.0–76.2	57.1–64.0	57.1–64.0	57.1–66.7
Mongoose (8 issues)	82.4	82.4	82.4	82.4
Axios (6 issues)	80.0	75.0	75.0	75.0

Three patterns emerge:

Pattern 1: Well-known libraries benefit most from docs. Express v4→v5 migration shows the starkest improvement: without docs, the baseline sometimes finds zero issues (the LLM doesn't know about removed methods like `app.del()` or changed signatures like `res.json(status, body)`). With docs, all five issues are consistently found.

Pattern 2: Popular libraries show diminishing doc benefit. For Stripe and Mongoose, the LLM's parametric knowledge is already strong—Stripe patterns like hardcoded API keys and deprecated `charges.create()` are well-represented in training data. Documentation doesn't hurt but provides less marginal improvement.

Pattern 3: Complex migrations show precision challenges. OpenAI v3→v4 involves extensive API restructuring. While docs improve recall to 100% (all 8 issues found), precision drops as the doc-augmented LLM generates additional findings—some valid concerns not in our ground truth, others genuine false positives. This suggests that richer documentation can sometimes "over-prompt" the model into flagging more potential issues.

5.3 Hallucination Analysis

We measure hallucination as findings referencing API names not present in the provided documentation (applicable only to conditions B/C/D where docs are available).

The hallucination rate ranges from 22.9% (raw docs) to 25.7% (structured docs). Counterintuitively, raw docs show a slightly lower hallucination rate. Investigation reveals this is because raw docs, being longer, contain more API name mentions that happen to match LLM-generated findings—not because the LLM is more accurate, but because the grounding check is more lenient with a larger doc index.

This highlights a limitation of our hallucination metric: it measures syntactic grounding ("does this API name appear in docs?") rather than semantic grounding ("is this finding correctly derived from the docs?"). Future work should develop more nuanced hallucination detection.

5.4 Efficiency Analysis

Table 4: Token usage and latency by condition.

	A	B	C	D
Input tokens	1,864	2,683	2,632	2,854
Duration (s)	15.7	23.4	22.0	21.4
F1 / 1K tokens	38.0	31.6	32.8	30.4

Structured retrieval (C) uses 2% fewer input tokens than raw docs (B) while achieving higher F1, making it the most token-efficient doc-augmented approach. The full pipeline (D) uses more tokens due to memory context but achieves the highest absolute quality.

6 Discussion

6.1 When Does Documentation Help?

Our results suggest documentation augmentation is most valuable when:

- The migration involves **API renaming or removal** (Express, OpenAI), where the LLM's parametric knowledge may be outdated or confused across versions;
- The library is **less popular** or has **recent changes** not well-represented in training data;
- The issues involve **subtle signature changes** (e.g., Express's `res.json(status, body)` → `res.status(200).json(body)`) that the LLM might not flag from training data alone.

Documentation provides less marginal benefit for well-known security anti-patterns (hardcoded API keys, missing webhook verification) that LLMs already flag reliably from training data.

6.2 Structured vs. Naive Retrieval

The 1.7 F1-point advantage of structured over naive retrieval, while modest, is achieved with fewer tokens and greater consistency. We attribute this to:

- **Section splitting** reduces noise by surfacing only relevant doc sections rather than entire documents;
- **API name extraction** creates an explicit index that enables precise grounding checks;
- **Diff analysis** highlights breaking changes, focusing the LLM's attention on the most critical migration details.

6.3 The Role of Memory

The memory layer's 0.3 F1-point contribution in our benchmark is modest because each fixture is reviewed in isolation without prior context. In production settings, we expect memory to provide larger benefits through:

- **Pattern accumulation:** Learning project-specific coding conventions and previously addressed deprecations;
- **False positive suppression:** Remembering that certain patterns are intentional in a specific codebase;
- **Cross-session consistency:** Maintaining coherent review standards across multiple review sessions.

Evaluating these longitudinal benefits requires a different experimental design, which we leave to future work.

6.4 Implications for LLM Agent Design

Our findings support the emerging consensus that LLM agents benefit from **tool augmentation** [15] and **retrieval grounding** [8]. Specifically for software engineering agents:

(1) **Version-specific context is critical.** Generic documentation retrieval is insufficient; agents need docs for the *specific versions* of libraries in the project under review.
(2) **Structured retrieval outperforms naive context stuffing.** Simply dumping documentation into the prompt is less effective than structured extraction and targeted injection.
(3) **Tool-use interfaces improve reliability.** Using structured output interfaces (Claude's tool-use, OpenAI's function calling) for JSON output eliminates parsing failures that plague free-text extraction.
(4) **Provider abstraction enables portability.** DocAware's multi-LLM support (Claude, GPT-4o, Gemini, Ollama) demonstrates that the doc-augmentation approach is model-agnostic—the benefits of structured retrieval are not tied to a specific LLM vendor.

7 Threats to Validity

Internal validity. Our ground-truth labels were manually created and may contain errors. The fuzzy matching criteria (3-line tolerance, suffix matching) may incorrectly match or miss findings. We mitigated this by having each fixture reviewed independently and by using conservative matching thresholds.

External validity. Our evaluation covers five JavaScript libraries. Results may not generalize to other languages, less popular libraries, or different types of code issues (e.g., logic errors, performance problems). The benchmark fixtures are deliberately constructed to contain known issues, which may not represent the distribution of issues in real-world projects.

Construct validity. Our hallucination metric measures syntactic grounding (API name presence in docs) rather than semantic correctness. A finding may reference a real API but still be incorrect in its assessment. Additionally, the F1 metric weights precision and recall equally, which may not match all use cases.

Reproducibility. LLM outputs are non-deterministic. We observed variation across runs (e.g., OpenAI fixture F1 ranging from 57.1% to 76.2% for the same condition). We report aggregated means across multiple runs to mitigate this, but individual results may vary.

8 Conclusion

We presented DocAware, a documentation-augmented AI agent for code review and API migration. Through controlled experiments across five library migration scenarios, we demonstrated that:

- Documentation augmentation improves F1 from 70.9% to 86.8% (22.4% improvement);
- Recall increases from 76.4% to 98.6%, meaning the system catches nearly all real API issues;
- Structured documentation retrieval outperforms naive RAG while using fewer tokens;
- A persistent memory layer provides modest additional improvement, with expected larger benefits in longitudinal use.

These results demonstrate that grounding LLM agents in version-specific, structured documentation significantly improves their reliability for software engineering tasks. As LLMs become more deeply integrated into development workflows, ensuring they operate on accurate, current information—rather than relying solely on training data—is essential for trustworthy AI-assisted development.

Future work. We plan to extend DocAware to additional languages (Python, Go, Rust), evaluate on larger real-world codebases, develop more nuanced hallucination metrics, conduct longitudinal studies to measure the memory layer's cumulative benefits, and benchmark across different LLM providers to compare how documentation augmentation impacts models of varying capability levels.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[2] Zhongxin Chen, Zhenchang Xing, Xin Xu, and Liming Zhu. 2023. On the Usage of Automated API Migration Tools. *Comput. Surveys* 55, 3 (2023), 1–38.

[3] Danny Dig and Ralph Johnson. 2006. Automated Detection of Refactorings in Evolving Components. In *European Conference on Object-Oriented Programming (ECOOP)*. 404–428.

[4] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[5] Marijn Haverbeke. 2023. Acorn: A Small, Fast, JavaScript-Based JavaScript Parser. https://github.com/acornjs/acorn.

[6] Kevin Jesse, Toufique Ahmed, Premkumar T. Devanbu, and Emily Morgan. 2023. Large Language Models and Simple, Stupid Bugs. *IEEE/ACM International Conference on Mining Software Repositories (MSR)* (2023), 563–575.

[7] Julia Lawall and Gilles Muller. 2018. Coccinelle: 10 Years of Automated Evolution in the Linux Kernel. In *USENIX Annual Technical Conference*. 601–614.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[9] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Janber, Le Luo, Ryan Golde, and Neel Sundaresan. 2022. Automating Code Review Activities by Large-Scale Pre-Training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 1035–1047.

[10] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. *arXiv preprint arXiv:2404.00971* (2024).

[11] Tung Thanh Nguyen, Hoan Anh Nguyen, Nam H. Pham, Jafar M. Al-Kofahi, and Tien N. Nguyen. 2009. Graph-Based Mining of Multiple Object Usage Patterns. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 383–392.

[12] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).

[13] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. 1–22.

[14] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590* (2023).

[15] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36.

[16] Rosalia Tufano, Arghavan Moradi Dakhel, Luca Pascarella, and Gabriele Bavota. 2024. Code Review Automation: Strengths and Weaknesses of the State of the Art. *IEEE Transactions on Software Engineering* 50, 2 (2024), 354–378.

[17] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2471–2484.

[18] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Wang, Zhiruo Jiang, and Graham Neubig. 2023. DocPrompting: Generating Code by Retrieving the Docs. In *International Conference on Learning Representations (ICLR)*.